



عملي

4

11

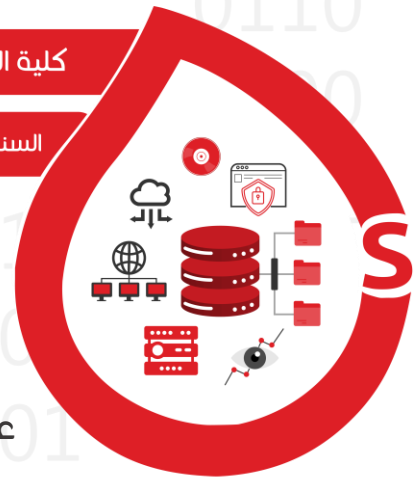
1650 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Non-Clustered index

عملي مشترك



RB Informatcs; 17/11/2025

قواعد معطيات ٢

Non-Clustered index

ذكرنا أهم الفروق بين الـ Clustered index و الـ Non-Clustered index في المحاضرة السابقة – كيف نُنشئه وكيفية إنشاؤه وتحدثنا عن بنيته وفي هذه المحاضرة سوف نتكلم عن الـ NON CLUSTERED INDEX

هل ستكون طريقة انشاءه تختلف عن انشاء الـ CLUSTERED INDEX ؟

يمكن انشاء الـ NON CLUSTERED INDEX بنفس طريقة انشاء الـ CLUSTERED INDEX أي :

```
CREATE NONCLUSTERED INDEX NAME ON
TABLE NAME(COLUMNS);
```

في الـ NON CLUSTERED INDEX يكون شكل الـ LEAF LEVEL بهذا الشكل :

(PRIMARY KEY OR RAW ID , KEY VALUE)

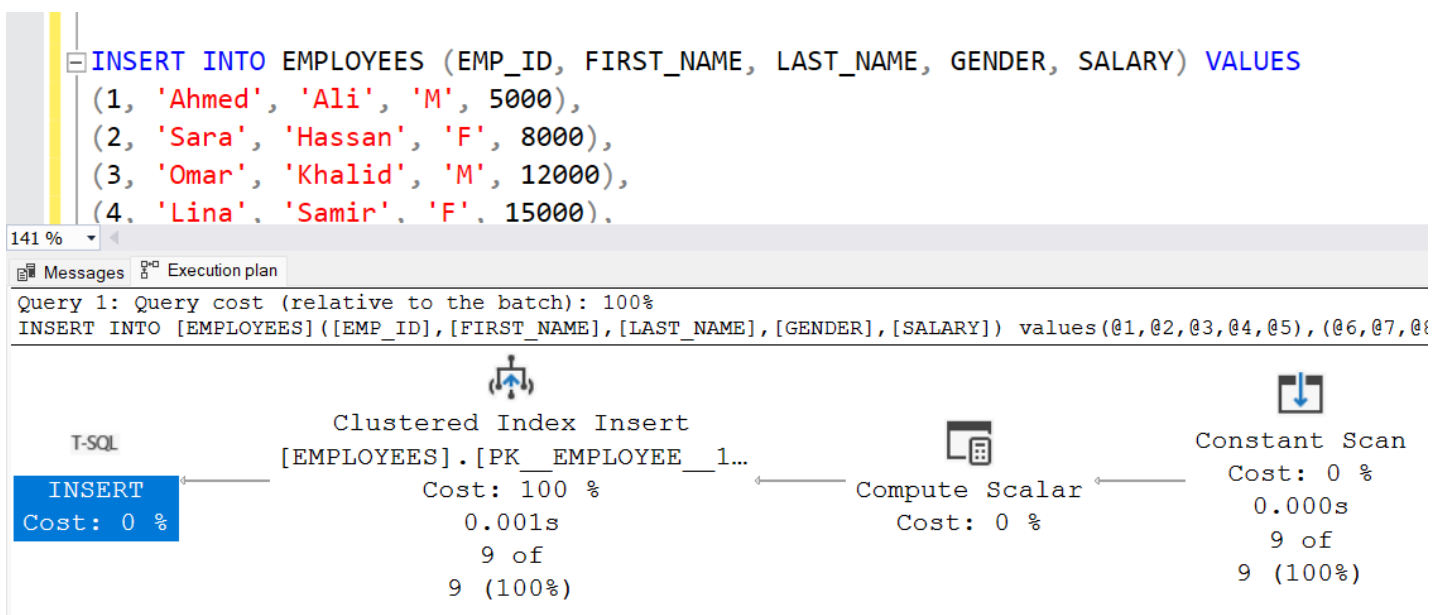
1. PRIMARY KEY: هو KEY يأخذ تلقائيا قيمة الـ ROW ID لـ الجدول الأساسي وهو رقم افتراضي يعطي لـ الـ RECORD عند انشائها ويكون العمل من خلاله الا اذا تم تحديد الـ PRIMARY KEY لـ الجدول الأساسي فيكون هناك أولوية له ف يستخدم ك مفتاح لـ الـ NON-CLUSTERED INDEX

2. VALUE: وهو القيمة التي يحملها المفتاح الذي نبني على أساسه الـ NON CLUSTERED INDEX

لنقم ببناء جدول لـ الموظفين وهو :

```
CREATE TABLE EMPLOYEES(
EMP_ID INT PRIMARY KEY,
FIRST_NAME VARCHAR(20),
LAST_NAME VARCHAR(20),
GENDER CHAR (50),
SALARY INT NOT NULL,
)
```

```
INSERT INTO EMPLOYEES (EMP_ID, FIRST_NAME, LAST_NAME, GENDER, SALARY) VALUES
(1, 'Ahmed', 'Ali', 'M', 5000),
(2, 'Sara', 'Hassan', 'F', 8000),
(3, 'Omar', 'Khalid', 'M', 12000),
(4, 'Lina', 'Samir', 'F', 15000),
(5, 'Youssef', 'Mahmoud', 'M', 7000),
(6, 'Nour', 'Fahmy', 'F', 20000),
(7, 'Tariq', 'Adel', 'M', 10000),
(8, 'Mona', 'Sami', 'F', 9000),
(9, 'Hassan', 'Amr', 'M', 11000)
```



نلاحظ أن تحديد الـ Id كـ Primary Key يكافئ بناء Clustered Index على أساس الـ Id

في الـ SQL Server يتم بناء الـ Clustered index على أساس الـ Id في حال لم يتم تحديد عمود للترتيب على أساسه. وهذه الحالة الافتراضية (Default).
على الرغم من عمل الـ Id مرتبة حسب الـ Data وهذا ما يفسر ظهور الـ insert بشكل غير مرتب.

سؤال: هل يمكننا بناء PK مع الـ non-Clustered index ؟

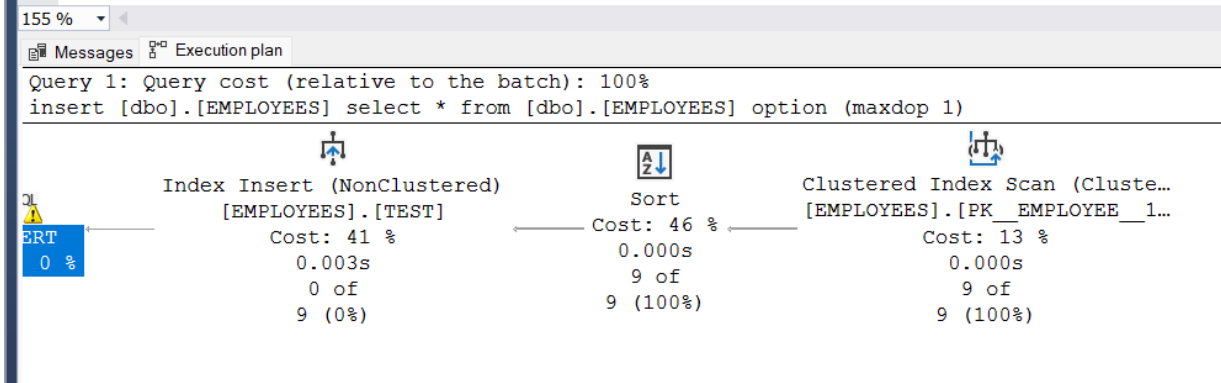
نعم يمكننا ذلك ولكن بشرط أن نعطيها ونحدد له أن:

Create Non-Clustered index primary key

سنقوم ببناء Non-Clustered index على أساس الـ Salary :

CREATE NONCLUSTERED INDEX TEST ON EMPLOYEES (SALARY)

CREATE NONCLUSTERED INDEX TEST ON EMPLOYEES (SALARY)



■ سؤال: ماذا حصل عندما قمنا بانشاءه ؟

بنى لدينا OBJ على أساس العמוד الذي حددنا وهو الراتب هنا وهو مكون من ثلاث مستويات (3 LEVELES)

■ سؤال: ماذا يحصل لـ Leaf node في حالة الـ Non-Clustered index ؟

Leaf node تصبح مكونة من:

key index: يمكن أن يكون إما Salary أو first name ... ويمكنه حمل Column أو أكثر من Column وهو في مثالنا (Salary).

Primary key, Row ID: يمكن أن يكون إما PK أو Row ID، حيث هما الاثنان يستطيعان جلب الـ records فإذا لم نكن نملك Primary key يقوم الـ engine بإعطاء Row ID بشكل عشوائي وغير مكرر لكل record والسبب في اعتماد احدهما هو كونهما يحملان قيم مميزة Unique على الـ Table وهو ما يساعد في جلب الـ Records.

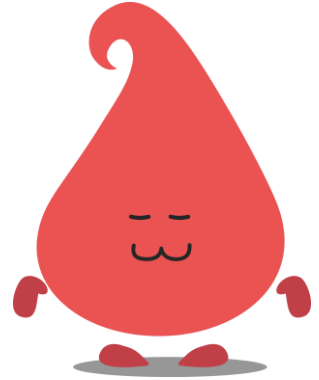
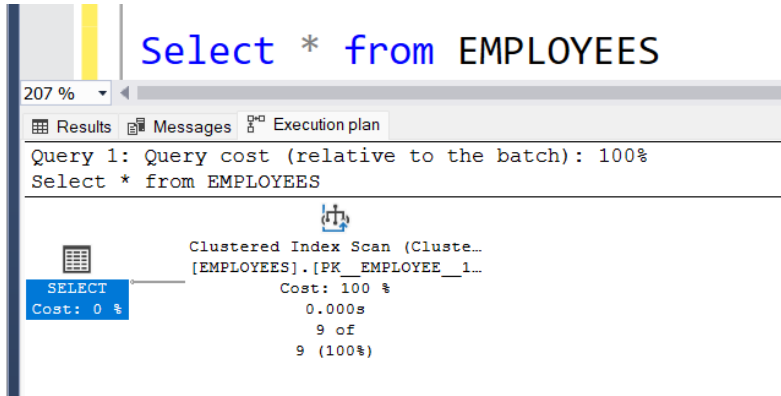
ملاحظة: لا يمكن اجتماع Key index + Row ID + PK في الـ Leaf node كقيم ثلاث، صحيح أنه يمكن اجتماع اثنتين ولكن الأساسيات هما الـ PK أو الـ Row ID

■ سؤال: في حال تم بناء Non-Clustered index كـ key للـ PK ثم عمل drop أساسي له ... فما الذي سيحصل ؟

في هذه الحالة الـ engine يتصرف بذكاء ويقوم بالتبديل تلقائياً من PK إلى Row ID لاستعادة المعلومات. ولكن في حال وجود عدد كبير من الـ records سيكلف كثير وقت، لذا ينبغي توخي الحذر.

■ بنينا سابقاً Non-Clustered على ال Salary مع وجود Clustered index على ال ID
عند تنفيذ الاستعلام:

Select * from EMPLOYEES



■ نلاحظ أنه قام بـ Clustered Scan ولم يتغير شيء.
ظهرت جميع ال Records الموجودة.

■ لماذا؟؟

يعني:
تريد كل الأعمدة – كل السجلات – كل الجدول كامل.

وحتى لو كان يوجد:

- Clustered index على ID
- Non-Clustered index على Salary
- index 20 غيرهم

النتيجة ستكون واحدة :

SQL Server لا يستطيع استخدام Non-Clustered Index لجلب كل الأعمدة.

لأن ال Non-Clustered index لا يوجد به جميع البيانات
يوجد به فقط:

- ال Key مثل (Salary).
- وال row ID أو ال PK .

لكن ما فيه بقية الأعمدة مثل

name – gender – department – salary

لذلك: SELECT* هو مجبر أن يرجع للجدول الأصلي (Clustered index)



■ سؤال ؟ لماذا ببعض الأحيان يظهر Non-Clustered Scan ؟

أن SQL Server أحياناً يقرر:

“إذا بداي أعمل Table Scan أو Non-Clustered Scan كلاهما نفس الكلفة تقريباً”.

فيستخدم

Non-Clustered index الموجود، ويعيد منه لا clustered عن طريق lookup .

أي أن السبب الحقيقي:

الاستعلام يطلب كل البيانات ولا يوجد فائدة حقيقية من أي index لذلك ال optimizer يستخدم أقرب خيار بدون فرق كبير بالكلفة.

لنقوم بالاستعلام مع استخدام ال WHERE:

Select * from emp where (salary > 7000)

Query 1: Query cost (relative to the batch): 100%

SELECT * FROM [EMPLOYEES] WHERE [salary]>@1

Clustered Index Scan (Clustered Index Scan)

[EMPLOYEES].[PK__EMPLOYEE__1...]

Cost: 100 %

0.000s

7 of

6 (116%)

SELECT

Cost: 0 %



نلاحظ في ال Execution plan أن ال Clustered index Scan ظهر لأنه ذهب مباشرة إلى Clustered index وقام بمسح البيانات وجلب ال Records المطلوبة. (salary > 7000).

ولكن يوجد طريق آخر: وهو أن يستخدم ال Non-Clustered index ثم يقوم بعملية تدعى lookup لجلب البيانات من ال Clustered index

■ كيف يتم العمل عندما يكون لدينا ال WHERE:

عندما نقوم بالاستعلام where سيكون لدى ال opt طريقين:

- إما يذهب إلى ال Clustered index ويأتي بالبيانات كلها.
- أو يستخدم ال Non-Clustered index وهنا سيقوم ب lookup لأن ال Non-Clustered لا يحتوي البيانات كلها، فقط يحتوي على salary وال PK الخاص به.

وعندما يقوم ب select سيأتي بباقي البيانات من ال Clustered index عن طريق lookup ال opt يحدد الطريق المناسب بحساب الكلفة ويأخذ الأقل

ماهي عملية ال LOOKUP ؟

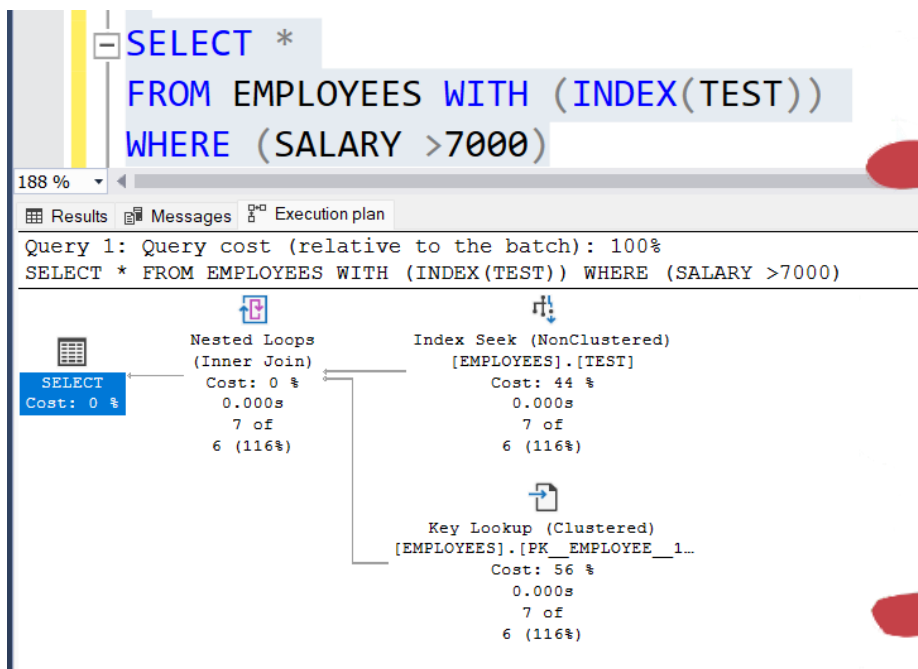
تحدث عملية Lookup في SQL Server عندما يستخدم المحرك فهرساً من نوع Non-Clustered Index للعثور على الصفوف المطلوبة، ولكن هذا الفهرس لا يحتوي على جميع الأعمدة التي يطلبها الاستعلام. في هذه الحالة يضطر SQL Server للعودة مرة أخرى إلى الجدول الأصلي أو إلى ال Clustered Index لجلب الأعمدة الناقصة، وهذه العملية تُسمّى Key Lookup إذا كان الجدول يحتوي Clustered Index أو RID Lookup إذا كان الجدول Heap.

تكون عملية ال Lookup مفيدة عندما يكون عدد الصفوف المطلوب جلبها قليلاً، لأن تكلفتها تكون منخفضة. ولكن إذا كانت نتيجة الاستعلام كبيرة، فإن تكرار عملية ال Lookup لكل صف قد يؤدي إلى بطء واضح في الأداء. ويمكن التخلص من هذه العملية عادةً عبر إنشاء ما يُسمّى Covering Index، وهو فهرس يحتوي على الأعمدة المستخدمة في شروط الاستعلام بالإضافة إلى الأعمدة المطلوبة في الإخراج، وذلك باستخدام جملة INCLUDE.

باختصار:

ال Lookup هو عملية يلجأ إليها SQL Server لجلب أعمدة ناقصة من الجدول عندما لا يغطي الفهرس المطلوب كل الأعمدة، وقد تكون مفيدة أو مكلفة حسب حجم البيانات

```
SELECT *
FROM EMPLOYEES WITH (INDEX(TEST))
WHERE (SALARY >7000)
```



INDEX التعليلة تقوم بإجبار ال opt على تنفيذ ال query باستخدام ال index المذكور اسمه.

■ سؤال ؟ ماهو عمل ال WITH هنا في الاستعلام ؟

تُستخدم عبارة (INDEX) WITH في SQL Server كنوع من التوجيه الإجباري (Query Hint) ، أي أنها تُلزم المُحرِّك (Optimizer) باستخدام فهرس معيّن أثناء تنفيذ الاستعلام، حتى لو كان المُحرِّك يرى أنّ فهرساً آخر أو طريقة تنفيذ مختلفة أكثر كفاءة.

أي (INDEX) WITH تعني:

“استخدم هذا الفهرس تحديداً ولا تستخدم أي فهرس آخر مهما كانت كلفته”.

تذكرة :

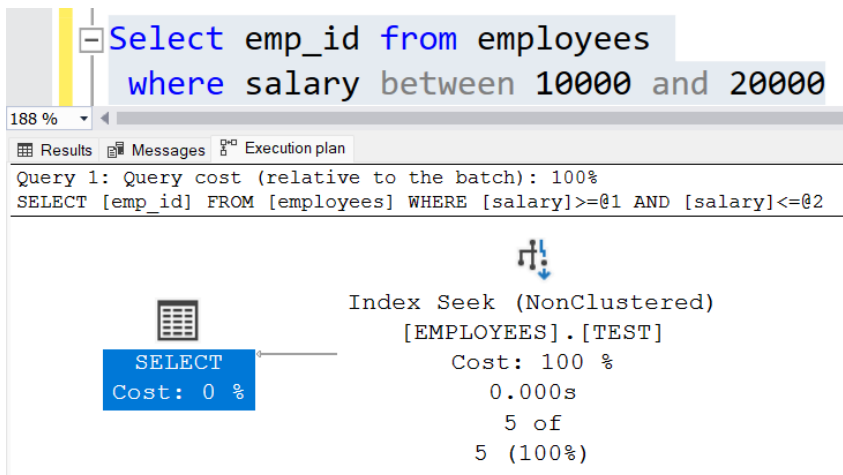
لدينا عمليتان أساسيتان:

1. Scan: يسمح لجميع ال data لأنه لا يوجد شيء مميز.

2. Seek: يمكننا الحصول على data بشكل مباشر لأنه يوجد شيء محدد.

مثال لتوضيح الفكرة

Select emp_id from EMPLOYEES
where salary between 10000 and 20000

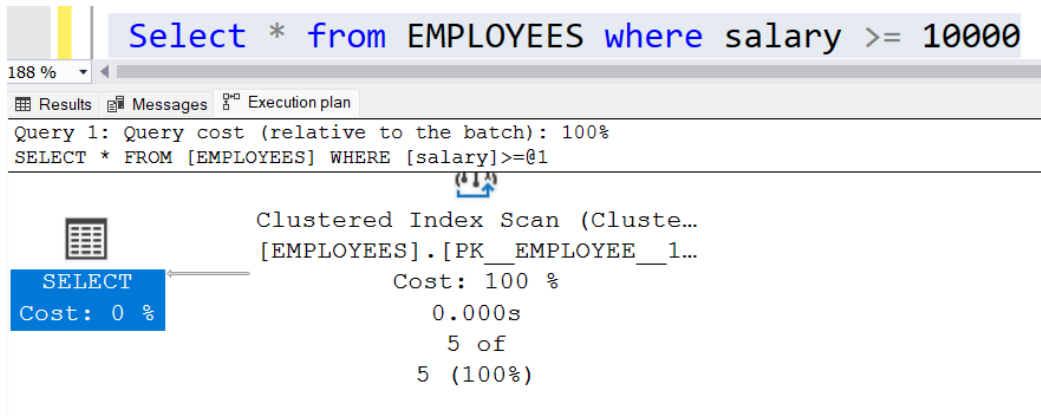


هنا نكون قد حصرنا ال records التي سنقرأ منها وبالتالي سيذهب Seek .

من عرف ما يطلب هان عليه ما يبذل



Select * from EMPLOYEES where salary >= 10000

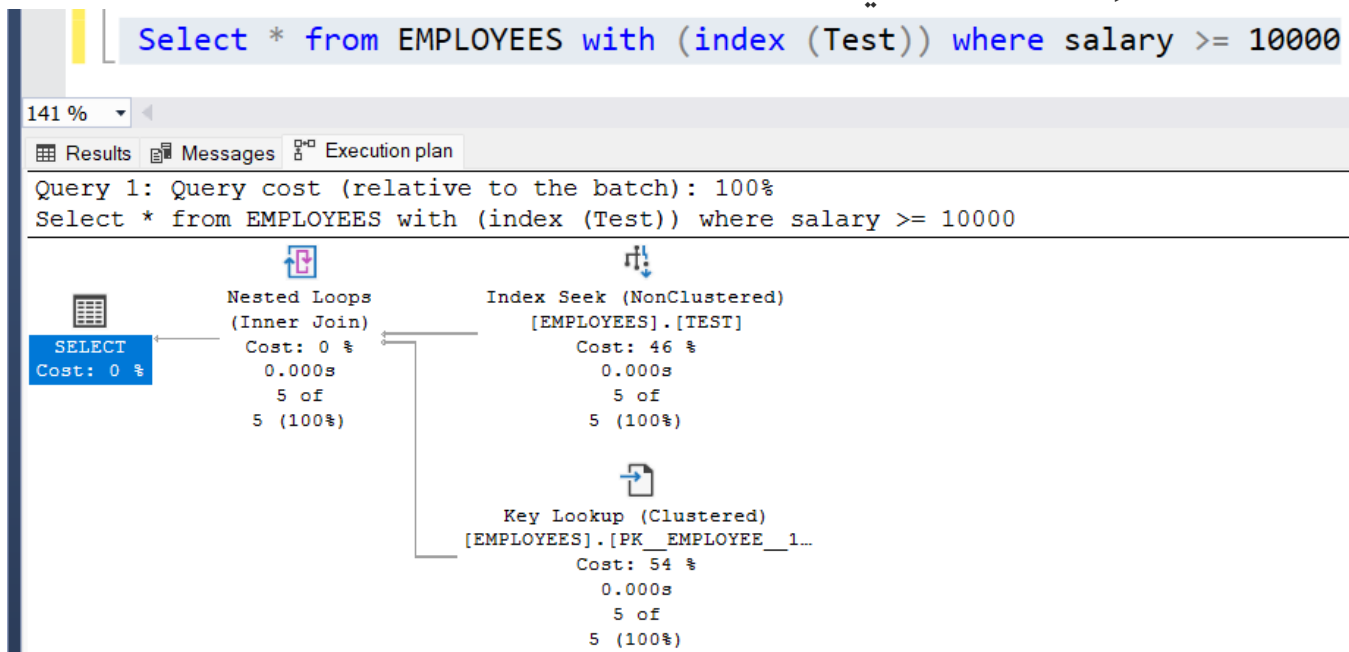


هنا سيقوم الـ opt بـ Clustered index scan لجلب النتيجة.

سؤال: ماذا حصل عندما نضيف WITH على الـ Query ؟

Select * from EMPLOYEES with (index (test3)) where salary >= 10000

سيقوم بـ index Seek ثم lookup لجلب باقي البيانات.



سؤال: هل هذه العملية تكلفتها أكبر من الـ Scan ؟

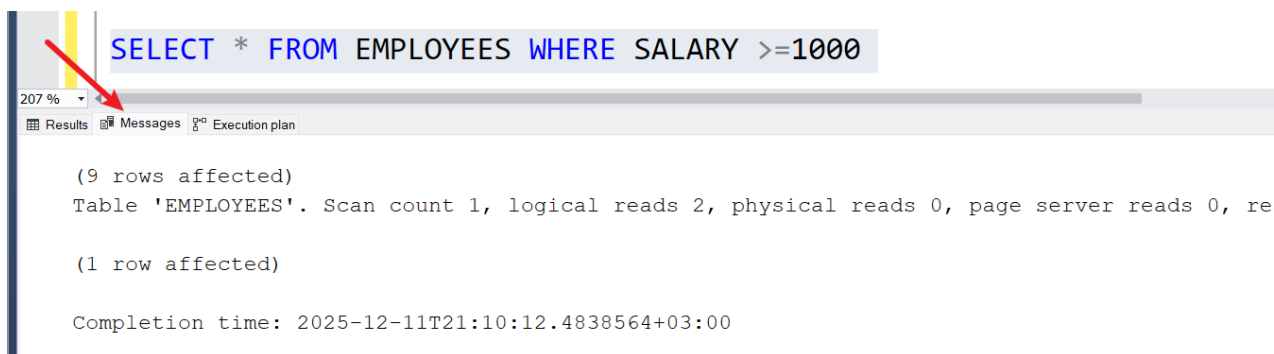
سنقوم برؤية Statistics input

(تعليلة الـ STATISTICS input هي تعليلة تقوم بعرض كم من مرة قام الـ optimizer بعمل الـ physical read وكم مرة قام بعمل الـ ep)

نقوم أولاً بكتابة التعليلة التالية وتنفيذها:

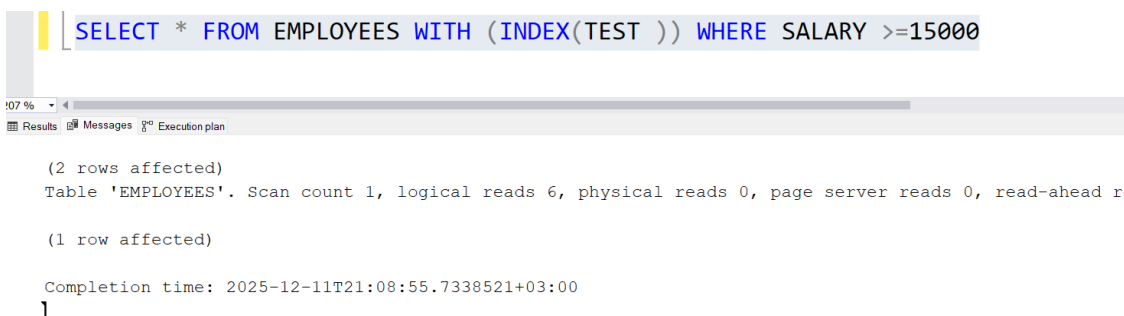
SET STATISTICS IO ON

```
SELECT * FROM EMPLOYEES WHERE SALARY >=1000
```



في المرة الثانية عندما اضفنا ال WITH :

```
SELECT * FROM EMPLOYEES WITH (INDEX(TEST )) WHERE SALARY >=15000
```



نلاحظ أنه قام بـ 6 logical read ... وبالتالي سيستغرق وقت وجهد أكبر.

سؤال: هل يجب أن نتدخل في عمل ال opt ؟

الجواب: لا.

الأغلب هو يختار الأفضل، لكن دورنا كمهندسين هو تهيئة البيئة المناسبة له مثل بناء Clustered index أو

Non-Clustered index.

سؤال: هل يمكن لنفس ال Query أن تُنفذ بطريقتين مختلفتين؟

نعم. حسب حجم ال data وحجم ال records

عندما تصبح ال data أكبر، قد يلجأ ال opt إلى خيار آخر وهو seek بال salary ثم يقوم بعملية lookup.

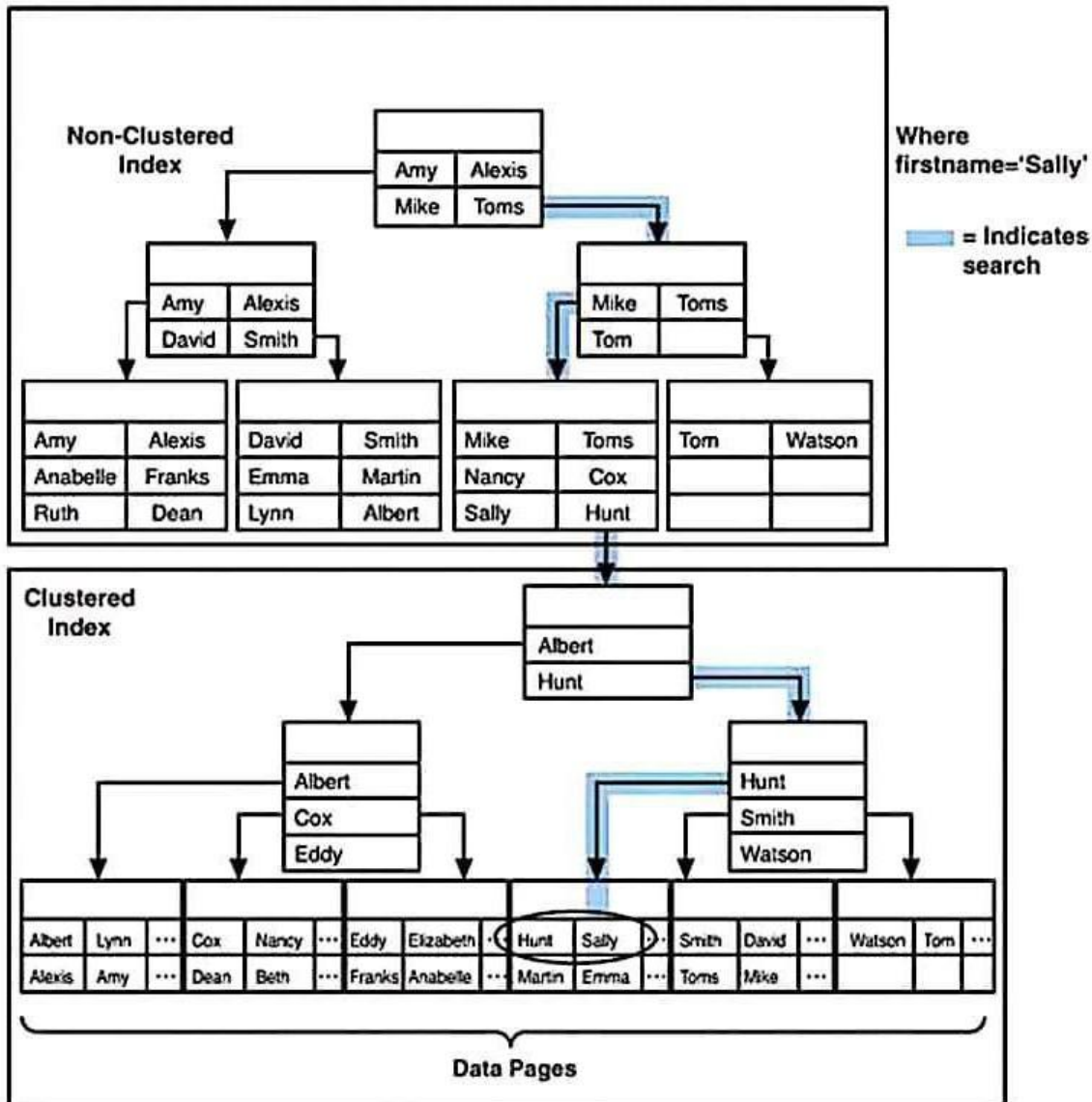
Never go to a doctor whose office plants

have died



ليس بالضرورة ان نستخدم ال NON CLUSTERED INDEX دائما لانه لا يحوي على كل الحقول

مثال :



■ كيف سيقوم ال NON CLUSTERED بعمل ال LOOKUP ؟

الجزء العلوي من المخطط هو ال NON CLUSTERED INDEX وهو مبني على الاسم الأول
الجزء السفلي من المخطط هو ال CLUSTERED INDEX وهو مبني على الاسم الأخير

مثال:

نصل الى SALLY من خلال ال NON CLUSTERED INDEX ثم نحصل على بقية المعلومات من ال CLUSTERED INDEX من خلال عملية ال LOOK UP

أي اذا كان لدينا 1000 SALLY سيصل الى ال NON CLUSTERED INDEX إلى جميع ال (SALLY) RECORDS في مرة واحدة ومن ثم سوف يقوم بعمل ال LOOKUP ل ال RECORDS التي اسمها SALLY ويجلب البيانات لا ننسى بان ال OPTIMIZER هو الذي يقوم بجلب الداتا من ال DATABASE بأسرع وافضل طريقة واقل كلفة .

جدول مقارنة بين ال Clustered Index وال Non Clustered Index

الخاصية / المعيار	Clustered Index	Non-Clustered Index
تعريف	فهرس يتم فيه ترتيب البيانات الفعلية للجدول.	فهرس منفصل عن البيانات الفعلية، يحتوي على مفتاح المؤشر و ROW ID أو PK.
عدد الفهارس المسموح بها على الجدول	1 فقط	يمكن وجود عدة فهارس غير متجمعة.
Leaf Node	يحتوي على الصفوف الفعلية للجدول.	يحتوي على Key Index + Pointer (PK أو ROW ID).
تأثير على INSERT/UPDATE/DELETE	يمكن أن يكون أثقل بسبب إعادة ترتيب البيانات.	أخف نسبياً، لكن يحتاج تحديث الفهرس عند تعديل العمود المفهرس.
السرعة في الوصول للبيانات	أسرع للوصول إلى البيانات بالكامل أو حسب ترتيب المفتاح.	أسرع للوصول إلى عمود محدد أو مجموعة بيانات محدودة، لكنه يحتاج Lookup لبقية الأعمدة.
مثال على الإنشاء	CREATE CLUSTERED INDEX IDX_EMP_ID ON EMPLOYEES(EMP_ID);	CREATE NONCLUSTERED INDEX IDX_EMP_SALARY ON EMPLOYEES(SALARY);
الاستخدام الأمثل	للاوصول السريع للبيانات حسب ترتيب العمود الرئيسي، وغالباً على PRIMARY KEY.	للاوصول السريع لأعمدة محددة في الاستعلامات المتكررة، وخاصة WHERE أو JOIN.
تأثير على SELECT	يمكن استخدامه مباشرة لأنه يحتوي على كل الأعمدة.	يحتاج Lookup ل جلب بقية الأعمدة لأنه لا يحتوي سوى على Key + PK/ROW ID.
تأثير على المساحة التخزينية	أقل نسبياً لأنه لا يكرر الصفوف.	أكثر لأنه يحتوي على فهرس منفصل يكرر المفتاح و Pointer.
نوع البحث (Scan/Seek)	Seek أسرع إذا الاستعلام يتوافق مع المفتاح، Scan إذا استعلام كامل.	Seek إذا العمود محدد، Scan إذا الاستعلام يتطلب كل الأعمدة أو شرط غير محدد.
مثال على الاستعلام	SELECT * FROM EMPLOYEES WHERE EMP_ID = 3;	SELECT * FROM EMPLOYEES WHERE SALARY > 7000;
Lookup	غير مطلوب.	مطلوب ل جلب الأعمدة غير الموجودة في الفهرس عند SELECT *.
تغيير المفتاح الأساسي	يحتاج إعادة إنشاء الجدول أو الفهرس.	يمكن تعديله بسهولة أكبر دون إعادة ترتيب الجدول.
عدد المستويات (Levels)	عادة 2-3 حسب حجم البيانات، Leaf Node = الصفوف.	عادة 2-3، Leaf Node = Key + PK/ROW ID فقط.
تأثير على الأداء عند عمليات Range	ممتاز عند البحث بنطاق على المفتاح.	يعتمد على عدد الصفوف المطلوبة، يمكن أن يكون Lookup مكلفاً إذا النتائج كبيرة.
الاعتماد على ROW ID	لا يحتاج، لأنه يحتوي على البيانات الفعلية.	يعتمد على PK أو Row ID ل جلب البيانات من الجدول.

النهاية